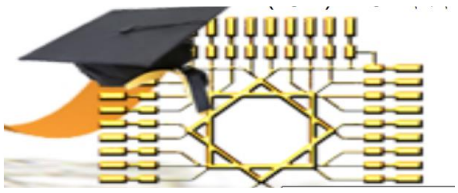
الموضوع: التسليم الثاني

اعداد: أنس نوفل

المشرف: ما.محمد بشار دسوقي

المقرر: اساسيات في النظم الموزعة



المعهد العالي للعلوم التطبيقية والتكنلوجيا

عميل HTTP غير متزامن ومجمع نتائج الـWorkers

التعديلات التي تم اجرائها على WebClient

في دالة sendTask قمت باضافة هذا الكود:

```
    */
    return sendTask(url, requestPayload, headers: null);
}
```

بدل ان نكرر كود ارسال الطلب وكل مرة نحتاج الى ارسال الطلب رح يصير عنا تكرار كبير بالكود فهي الطريقة هي الانسب لحل هي المشكلة.

بناء طلب HTTP:

```
java.net.http.HttpRequest.Builder requestBuilder =
        java.net.http.HttpRequest.newBuilder()
                .uri(java.net.URI.create(url))
                .POST(java.net.http.HttpRequest.BodyPublishers
                        .ofByteArray(requestPayload));
```

هنا قمنا بتحديد ثلاثة اشياء:

1- عنوان الخادم الذي سيرسل اليه الطلب
2- طريقة الطلب Post
3- جسم الطلب, الذي يحوي البيانات الموجودة في requestPayload

استخدمنا ofByteArray لان الدالة تستقبل البيانات على شكل مصفوفة من البايتات

اضافة header اختياري:

```java
if (headers != null && !headers.isBlank()) {
    int separator = headers.indexOf(':');

    if (separator <= 0) {
        throw new IllegalArgumentException(
                "Invalid header format. Expected Name: Value");
    }

    String headerName = headers.substring(0, separator).trim();
    String headerValue = headers.substring( beginIndex: separator + 1).trim();

    requestBuilder.header(headerName, headerValue);
}
```

عندما نستقبل النص: X-Debug: true

نفصل اسم الرأس عن قيمته ونضيفه الى الطلب واذا لم يمرر اي رأس يرسل الطلب دون الرأس مخصص.

إرسال الطلب بصورة غير متزامنة

```java
return client.sendAsync(
                request,
                java.net.http.HttpResponse.BodyHandlers.ofString())
        .thenApply(java.net.http.HttpResponse::body);
```

sendAsync: ترسل الطلب بصورة غير متزامنة وتعيد CompletableFuture

BodyHandlers.ofString: تجعل جسم الاستجابة متوفر كنص

thenApply: تحول الاستجابة الى جسمها الاصلي باستخدام body

التعديلات التي تم اجرائها على WebClient

الدالة الأولى تستدعي الدالة الثانية

```
return sendTasksToWorkers(workersAddresses, tasks, headers: null);
```

هذه الدالة مخصصة لارسال المهام دون رؤوس مخصصة بدل ما نكتب منطق الإرسال مرتين نستدعي الدالة الثانية ونمرر null في مكان headers

التحقق من صحة المدخلات

```
if (workersAddresses == null || tasks == null) {
    throw new IllegalArgumentException(
            "Worker addresses and tasks must not be null");
}

if (workersAddresses.size() != tasks.size()) {
    throw new IllegalArgumentException(
            "The number of workers must match the number of tasks");
}
```

قمنا باضافة قائمة لعناوين العمال وقائمة أخرى للمهام، كما انه يجب ان يكون لكل مهمة عنوان عامل مقابل. لذا اذا كان هناك مهمة بدون عنوان عامل مخصص لها تظهر عندها رسالة تحذيرية بدل ان يظهر خطأ لاحقاً.

انشاء قائمة لتخزين الطلبات غير المتزامنة

```
List<CompletableFuture<String>> futures = new ArrayList<>();
```

ارسال كل مهمة الى العامل المقابل

```
for (int i = 0; i < tasks.size(); i++) {
    String address = workersAddresses.get(i);
    byte[] payload = tasks.get(i).getBytes(java.nio.charset.StandardCharsets.UTF_8);

    CompletableFuture<String> future;

    if (headers == null || headers.isBlank()) {
        future = webClient.sendTask(address, payload);
    } else {
        future = webClient.sendTask(address, payload, headers);
    }

    futures.add(future);
}
```

هنا في كل loop من الحلقة:

1- نأخذ عنوان العامل من القائمة

2- نحول المهمة من string الى byte لأن WebClient يستقبل البيانات بهذا النوع

3- نختار الدالة المناسبة حسب وجود الرؤوس المخصصة

4- نضيف CompletableFuture الناتج إلى القائمة

استخدام sendTask غير المتزامنة يسمح ببدء الطلبات تباعاً دون انتظار انتهاء الطلبات التي بدأت أولاً.

انتظار انتهاء جميع الطلبات:

```
CompletableFuture.allOf(
        futures.toArray(new CompletableFuture[0])
).join();
```

هنا نستخدم allOf لانشاء كائن يمثل جميع الطلبات ثم join لانتظار انتهائها.

ملاحظة: يجب التنويه الى انه إذا فشل أحد الطلبات، فقد ينتهي `join` باستثناء بدل إرجاع قائمة نتائج مكتملة. يمكن معالجة هذا الامر

جمع النتائج بالترتيب الصحيح:

```
List<String> results = new ArrayList<>();

for (CompletableFuture<String> future : futures) {
    results.add(future.join());
}

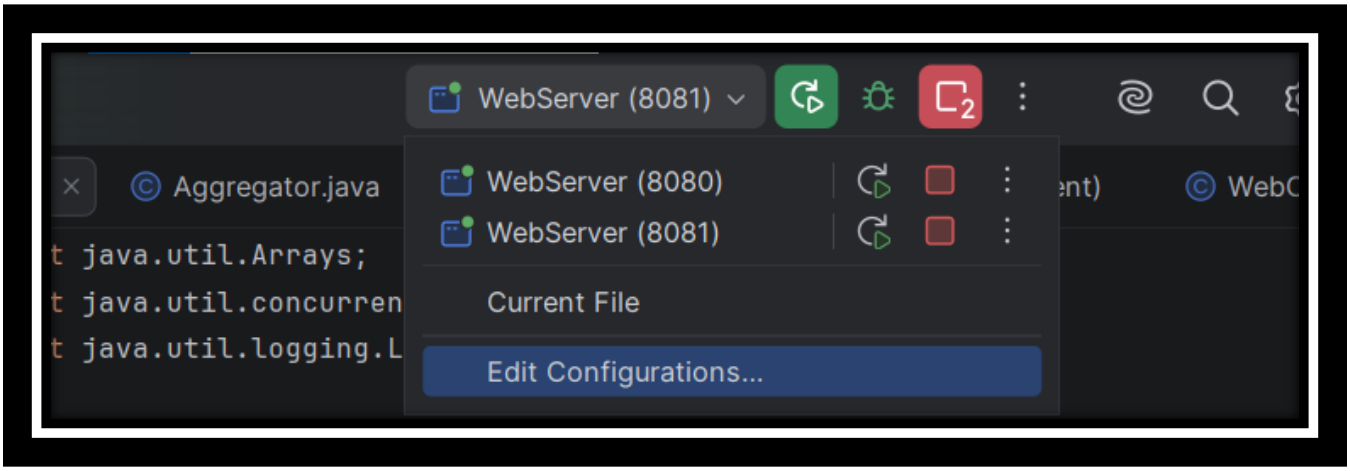
return results;
```

بعد اكتمال كل الطلبات نقرا النتائج بالترتيب الذي قمنا بتخزين CompletableFuture.

لا نعتمد على ترتيب وصول الاستجابات بل نعتمد على ترتيب القائمة لذلك تبقى نتيجة العامل الأول في الموضع الأول ونتيجة العامل الثاني في الموضع الثاني حتى لو وصل الرد الخاص بالعميل الثاني قبل العميل الأول.

الاختبارات والتحقق:

قمت بتشغيل الخادمين على البوابتين 8080,8081 على الترتيب من اجل اجراء الاخبار.



بعد تشغيل Application ظهر هذا الخرج الموضح في الصورة التالية

```
C:\Users\Anas\.jdks\openjdk-27.0.1\bin\java.exe "-javaagent:C:\Program
Result of the multiplication is 2000

Result of the multiplication is 864197525891781455166270000000000000

Result of the multiplication is 2000

Result of the multiplication is 864197525891781455166270000000000000


Process finished with exit code 0
```

السطر الاول من الخرج تحت :c هنا نرسل 10و200 فيحسب الخادم حاصل ضرب العددين. وهذا يدل على ان جسم الطلب وصل بنجاح الى الخادم وعولج بطريقة صحيحة.

المهمة الثانية تحوي على ثلاثة اعداد كبيرة، يقوم الخادم بحساب حاصل ضربها ويعطينا النتيجة

أما السطر الثالث والرابع فهما تكرار للسطرين الاول والثاني والسبب هو ان Application.java يقوم بتنفيذ مجموعتين من الطلبات الاولى دون الرأس المخصص، اما الثانية مع X-Debug: true

تحسينات على كود Application.java

1- التأكد من أن الخادم استقبل الترويسة X-Debug: true وأعاد ترويسة الاستجابة X-Debug-Info

```
HttpClient client = HttpClient.newHttpClient();

HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create(WORKER_ADDRESS_1))
        .header( name: "X-Debug",  value: "true")
        .POST(HttpRequest.BodyPublishers.ofString(task1))
        .build();

HttpResponse<String> response = client.send(
        request,
        HttpResponse.BodyHandlers.ofString()
);
```

أنشأت HttpClient لإرسال طلب Http ثم جهزت الطلب الموجه الى الخادم الأول ومن ثم اضفت ترويسة X-Debug: true وبعدها نقوم بارسال المهمة 200,10 باستخدام Post وتُستقبل الاستجابة في HttpResponse<String>

```
System.out.println("Response body: " + response.body());

System.out.println(
        "X-Debug-Info: " +
                response.headers().firstValue( name: "X-Debug-Info")
                        .orElse( other: "Header not found")
);
```

هنا نبحث عن ترويسة X-Debug-Info في استجابة الخادم

2- التأكد من أن النظام يتعامل مع اختلاف عدد المهام مع عدد الخوادم بطريقة صحيحية

هنا نمرر عنواني خادمين ومهمة واحدة فقط

```
try {
    aggregator.sendTasksToWorkers(
            Arrays.asList(WORKER_ADDRESS_1, WORKER_ADDRESS_2),
            Arrays.asList(task1)
    );

    System.out.println("ERROR: Mismatched input was not rejected.");

} catch (IllegalArgumentException e) {
    System.out.println("Expected error: " + e.getMessage());
}
```

```
C:\Users\Anas\.jdks\openjdk-27.0.1\bin\java.exe "-javaagent:C:\Program
=== Test 1: Without custom headers ===
Result of the multiplication is 2000

Result of the multiplication is 864197525891781455166270000000000000

=== Test 2: With custom headers ===
Result of the multiplication is 2000

Result of the multiplication is 864197525891781455166270000000000000

=== Test 3: Check X-Debug-Info response header ===
Response body: Result of the multiplication is 2000

X-Debug-Info: Operation took 107500 ns
=== Test 4: Mismatched workers and tasks ===
Expected error: The number of workers must match the number of tasks

Process finished with exit code 0
```

الخرج في Test 3 يؤكد أن الاستجابة تضمنت معلومة تشخيصية. لكن الزمن المقاس هنا ليس بالضرورة زمن الرحلة الكاملة عبر الشبكة؛ يعتمد ذلك على المكان الذي يبدأ فيه الخادم القياس وينهيه.

أما خرج Test 4 فهو بسبب ان عدد المهام لا يساوي عدد عناوين العمال